

University of Macedonia
Department of Applied Informatics

1st Laboratory Assignment

Neural Networks in Computer Vision

Course: Neural Networks

Academic Year: 2025 – 2026

Student Name: Apostolos Sinioris

Student ID: ics24123

Supervisor: Prof. Eftychios Protopapadakis

May 23, 2026

Contents

1	Introduction	3
2	Datasets Description	4
2.1	Data Characteristics and Preprocessing	4
2.1.1	The MNIST Dataset	4
2.1.2	The Fashion-MNIST Dataset	5
2.2	Data Visualization	6
3	Proposed Methodology	8
3.1	Neural Networks (NN)	8
3.2	Model Architectures	9
3.2.1	Deep Neural Network (DNN)	9
3.2.2	Convolutional Neural Network (CNN)	12
3.3	Transfer Learning Methodology	15
4	Training Process and Evaluation Strategy	18
4.1	Task 1 - The MNIST Dataset	18
4.2	Task 2 - The Fashion-MNIST Dataset	19
4.2.1	Training from Scratch (The Baseline)	19
4.2.2	Transfer Learning Approach	19
5	Experimental Results	21
5.1	Part 1: Architecture Comparison (MNIST)	21
5.2	Part 2: Architecture Comparison (Fashion-MNIST)	22
6	Conclusions	23

List of Figures

1	Sample image from the MNIST dataset showcasing handwritten digits. . .	6
2	Sample images from the Fashion-MNIST dataset, illustrating the variety and complexity of the clothing classes.	7
3	Biological vs Artificial Neuron	8
4	Simple Neural Network	9
5	Sample of a DNN	10
6	The softmax function takes a score vector s and a value of the softmax-parameter as input and returns a probability vector.	11
7	Architecture of the Proposed Deep Neural Network (DNN).	12
8	A simple CNN architecture, comprised of just five layers	12
9	Architecture of the Proposed Convolutional Neural Network (CNN) highlighting the feature map dimensions at each stage.	15
10	Traditional/Classical ML vs Transfer Learning	16
11	Average Accuracy and F1-Score of the DNN and CNN architectures on the MNIST test set across 6 folds.	21
12	Performance comparison between models trained from scratch and models utilizing Transfer Learning on the Fashion-MNIST test set.	22

1 Introduction

The rapid advancement of Deep Learning has significantly transformed the field of Computer Vision, particularly in tasks such as image classification and pattern recognition. Artificial neural networks are now capable of automatically extracting complex spatial features from images, achieving a performance that often rivals human vision.

The primary subject of this laboratory assignment is the practical application of deep learning models for image classification tasks. Specifically, the report focuses on the famous MNIST dataset, a standard benchmark in the machine learning community, which consists of grayscale images of handwritten digits.

The main objectives of this assignment are the following:

- **Architecture Design:** To design, implement, and compare two fundamentally different neural network architectures: a standard Deep Neural Network (DNN) and a Convolutional Neural Network (CNN).
- **Robust Evaluation:** To utilize the Stratified K-Fold cross-validation technique, ensuring that the training process is objective and the results are not influenced by a lucky data split.
- **Performance Analysis:** To evaluate both models using multiple classification metrics, including Accuracy, Precision, Recall, and F1-score, in order to determine the most effective architecture for computer vision tasks.
- **Transfer Learning Applicability:** To investigate the concept of transfer learning by taking the pre-trained models from the first task and adapting them to a new, but structurally similar dataset (Fashion-MNIST). The goal is to determine if prior knowledge extracted from handwritten digits can accelerate or improve the learning process for classifying articles of clothing.

Through this comparative study, we aim to demonstrate why Convolutional Neural Networks (CNNs) are generally preferred over standard dense networks when dealing with two-dimensional spatial data like images.

2 Datasets Description

In this section, we will describe the datasets which are used to train and test the model.

MNIST Dataset

The MNIST dataset is a widely used benchmark in Machine Learning for handwritten digit recognition. It contains preprocessed handwritten digit images derived from the original NIST dataset, making it suitable for research and experimentation. [1]

The dataset was created using handwritten digit datasets from the National Institute of Standards and Technology (NIST). It is widely used to learn the core fundamentals of Machine Learning and Pattern Recognition. Its small size is one of the advantages that makes it popular in the field of Machine Learning.

Fashion-MNIST Dataset

In the second part of the assignment, the Fashion-MNIST dataset is used. Fashion-MNIST is a dataset of Zalando's article images—consisting of a training set of 60,000 examples and a test set of 10,000 examples. [2]

2.1 Data Characteristics and Preprocessing

2.1.1 The MNIST Dataset

It consists of 70.000 grayscale images in total. For the training purposes, 60.000 of them are used. The other 10.000 are used for testing. [3]

- They have a fixed resolution of 28×28 pixels
- Each image contains 784 numerical features (28×28).
- Each image comes with a digit label (integers in range 0-9)

The data set was imported into the Google Colab notebook using the Python modules Tensorflow and keras.datasets.

```
1 # Import core modules
2 from tensorflow import keras
3 from keras.datasets import mnist
```

Listing 1: Importing libraries.

Then we load the data, print the original shape and we proceed with the normalization in the range of [0,1].

```
1 # 1. Load the data, split between train and test sets
2 # The MNIST dataset already provides a split of 60,000 for training and
3 # 10,000 for testing
4 (x_train, y_train), (x_test, y_test) = mnist.load_data()
5 # x is the input data
6 # y is the labels
7
8 # Print the original shape to verify (expecting 60000 images of 28x28
9 # pixels)
10 print('Original train shape:', x_train.shape)
11 print('Original test shape:', x_test.shape)
12
13 # 2. Normalize the data in range [0, 1]
```

```

12 # A grayscale image has pixel values in [0, 255]
13 x_train = x_train / 255.0
14 x_test = x_test / 255.0

```

Listing 2: Loading data and normalizing.

Data Normalization is a crucial preprocessing technique that scales numerical features to a fixed range $[0,1]$ to ensure that features with large magnitudes do not dominate models.

We proceed with the One-hot encoding technique that converts categorical data into a binary format (0s and 1s) by creating new binary columns for each unique category.

Example: $3 \rightarrow [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$

```

1 # 4. Do a one-hot-encoding for the outputs (labels)
2 # We have 10 classes (digits 0-9)
3 y_train = keras.utils.to_categorical(y_train, 10)
4 y_test = keras.utils.to_categorical(y_test, 10)

```

Listing 3: One-hot encoding.

2.1.2 The Fashion-MNIST Dataset

It shares the exact same image size (28×28 pixels), format (grayscale), and dataset splits (60,000 training images and 10,000 testing images). However, instead of handwritten digits, it contains images of various clothing articles. The images are categorized into 10 distinct classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

The data set was imported using the Python modules Tensorflow and keras.datasets.

```

1 from tensorflow import keras
2 from keras.datasets import fashion_mnist

```

Listing 4: Importing libraries

We load the data via the `fashion_mnist.load_data()` method and print the original shape.

```

1 # 1. Load the Fashion-MNIST dataset
2 (x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
3
4 print('Original Fashion-MNIST train shape:', x_train.shape)
5 print('Original Fashion-MNIST test shape:', x_test.shape)

```

Listing 5: Data loading

We normalize the data in the range of $[0,1]$, add a channel dimension and proceed to the One-Hot Encoding technique.

```

1 # 2. Normalize the data in range [0, 1]
2 x_train = x_train / 255.0
3 x_test = x_test / 255.0
4
5 # 3. Add a channel dimension (needed for CNNs)
6 x_train = np.expand_dims(x_train, axis=-1)
7 x_test = np.expand_dims(x_test, axis=-1)
8
9 # Save the original labels before One-Hot Encoding (for K-Fold)

```

```

10 y_train_original = y_train.copy()
11 y_test_original = y_test.copy()
12
13 # 4. Do a one-hot-encoding for the outputs (10 classes of clothes)
14 y_train = keras.utils.to_categorical(y_train, 10)
15 y_test = keras.utils.to_categorical(y_test, 10)

```

2.2 Data Visualization

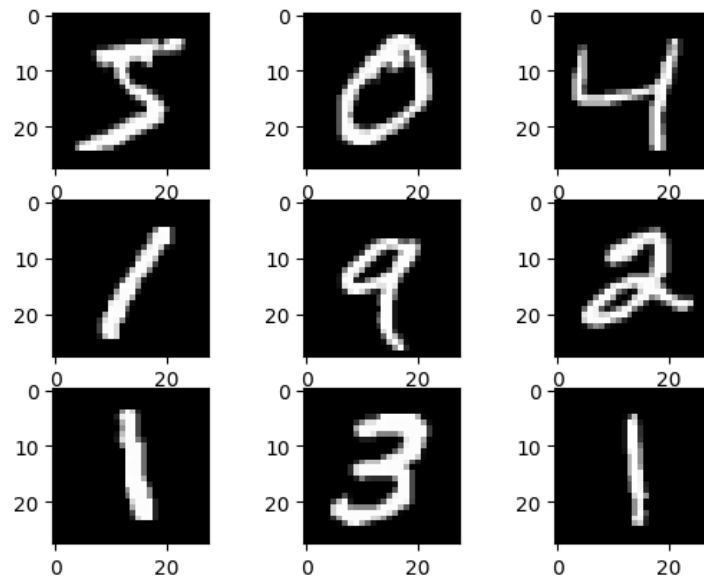


Figure 1: Sample image from the MNIST dataset showcasing handwritten digits.

As shown in Figure 1 the digits vary in thickness and style. Our job is to train 2 different Neural Networks (the DNN and the CNN) based on those images and decide which NN provides the best result.



Figure 2: Sample images from the Fashion-MNIST dataset, illustrating the variety and complexity of the clothing classes.

To better understand the nature of the new data, Figure 2 displays a random sample of twenty-five images from the dataset. As we can see, the pictures are in a small 3x3 grid, and they are completely grayscale. Below each image, there is a text label showing its exact category, such as "Ankle boot", "T-shirt/top", or "Dress". This simple visual check makes it clear that clothing items have much more detail and different shapes compared to plain handwritten digits, making the computer's job to recognize them much harder.

3 Proposed Methodology

In this section, we will present all the methodologies that were used to build the model. We used, Python as the programming language and Google Colab as the platform to test and run the model.

Before we proceed to describe the Model Architectures, we shall make a brief introduction about Neural Networks, in general.

3.1 Neural Networks (NN)

What is a Neural Network? Think of a human brain. Neural networks are models of the brain and the nervous system. They work in a similar manner like our brain and process information in the same way. Their principles are very simple but have complex behaviors. [4]

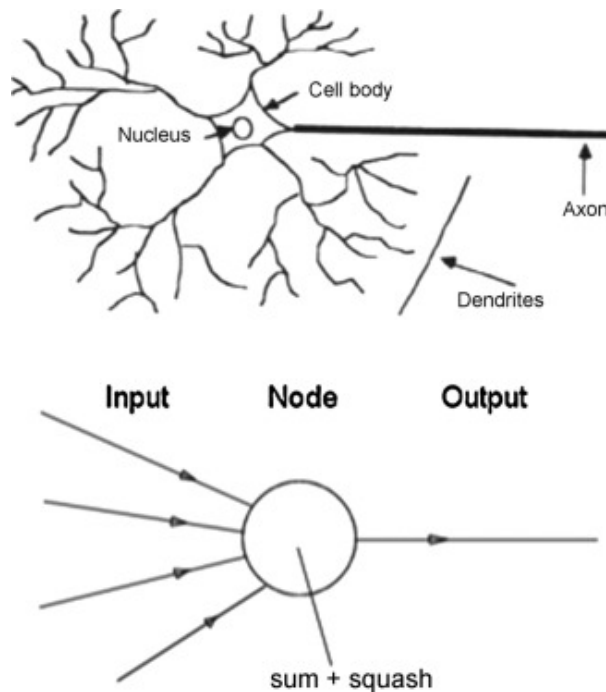


Figure 3: Biological vs Artificial Neuron

Our biological neurons (or neural cells) consist of dendrites that receive electrical signals, a cell body that processes them, and an axon that transmits the output to other connected neurons. Inspired by this biological mechanism, **Artificial Neural Networks (ANNs)** are built using interconnected artificial neurons, commonly referred to as nodes. These nodes are organized into distinct layers: an **Input layer** that receives the initial raw data (such as image pixels), one or more **Hidden layers** that perform complex mathematical transformations to extract patterns, and an **Output layer** that produces the final prediction or classification.

The connections between these artificial neurons are assigned mathematical values called **weights**, which determine the strength and importance of the signal passed from one node to the next. During the training process, the network "learns" by making predictions,

evaluating its mistakes using a loss function, and continuously adjusting these weights. This optimization is typically achieved through an algorithm called **Backpropagation**, which allows the network to minimize its errors and improve its accuracy over time.

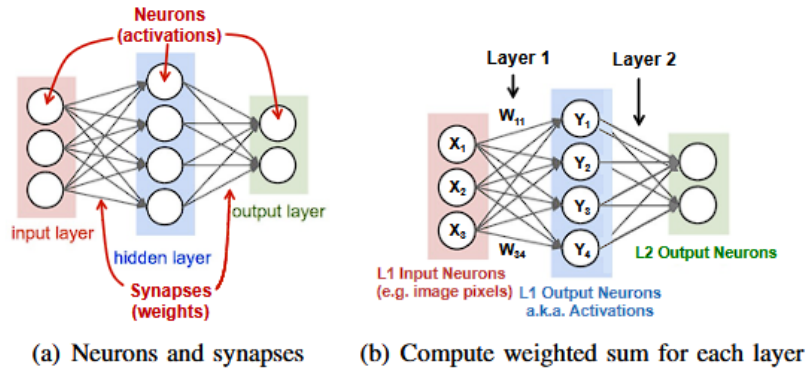


Figure 4: Simple Neural Network

Figure 4 shows a diagrammatic picture of a computational neural network. The neurons in the input layer receive some values and propagate them to the neurons in the middle layer of the network, which is also frequently called a ‘hidden layer’. The weighted sums from one or more hidden layers are ultimately propagated to the output layer, which presents the final outputs of the network to the user. [5]

3.2 Model Architectures

3.2.1 Deep Neural Network (DNN)

DNNs stands for **Deep Neural Networks**. They are used for many **Artificial Intelligence (AI)** tasks such as computer vision, speech recognition and robotics. They deliver state-of-the-art accuracy on many AI tasks, but this comes with a high GPU cost. GPUs are most likely used because they are specifically designed to handle complex matrix multiplications. [5]

Image classification like our project, is a common application of DNNs.

DNNs are capable of learning more complex and high-level features than traditional NNs. They usually contain more than 3 hidden layers, which distinguishes them from the traditional NNs. More hidden layers, enable the network to learn increasingly complex representations of the input data, capturing hierarchical patterns and features.

In a DNN every layer of nodes trains on a different set of features based on the previous layer’s output. So, the more you go into the net, the more you can advance into it, and the more complex features the nodes can recognize. This is because it recombines and learns from the features of the previous layer. This is what is called feature hierarchy. [4]

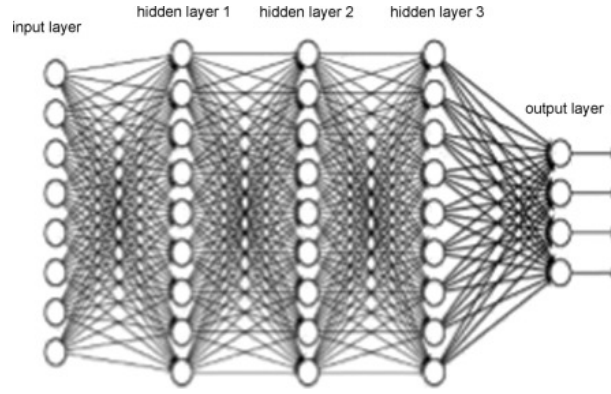


Figure 5: Sample of a DNN

In Figure 5, we see a DNN which contains 3 hidden layers. The input layer does not count as a hidden layer. Every neuron is connected with all the neurons of the next layer.

Here, we are going to show the DNN model we implemented as part of this project. Sample of code will be represented and followed by an explanation.

Flatten Layer

```

1 # 2. Flatten layer: Converts the 28x28 2D image into a flat 1D array of
  # 784 pixels
2 # DNNs (unlike CNNs) require the data to be in a single straight line.
3 model.add(Flatten(input_shape=(28, 28, 1)))

```

Listing 6: Flatten layer.

This is our first layer which is a **Flatten Layer**. Its purpose is to convert the 28x28 2D image from the input layer into a flat 1D array of 784 pixels. DNNs require the data to be in a single straight line (like a vector).

First Hidden Layer

```

1 # 3. First Hidden Layer (Dense means fully connected)
2 # Rule 2b: Use 'relu' activation function
3 model.add(Dense(128, activation='relu'))

```

Listing 7: First Hidden Layer

This is the first hidden layer in our DNN model. The **Dense** layer is the fundamental building block of a neural network. It is fully connected, which means that every neuron in a Dense layer receives input from every neuron in the previous layer. The first parameter is the number of neurons. The second parameter is the activation function. We use *ReLU* here.

ReLU (Rectified Linear Unit) is the most popular activation function used in Deep Learning. *ReLU* formula shows that if the input is less than 0 ($x < 0$), then the output is equal to 0. If the input is greater than 0 ($x > 0$), then the output equals to the input. [6]

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Dropout Layer

```
1 # 4. Dropout Layer
2 # Rule 2c: Use at least one dropout layer. We drop 20% of neurons to
  prevent overfitting.
3 model.add(Dropout(0.2))
```

Listing 8: Dropout Layer

A Dropout Layer is added to prevent the model from Overfitting. Overfitting happens when a model learns too much from the training data and it is unable to adjust to new data.

Second Hidden Layer

```
1 # 5. Second Hidden Layer
2 # Again using 'relu' as requested by the assignment
3 model.add(Dense(64, activation='relu'))
```

Listing 9: Second Hidden Layer

Just like the First Hidden Layer, the activation function is *ReLU*. We use 64 neurons, instead of 128.

Output Layer

```
1 # 6. Output Layer
2 # We have 10 digits (0-9), so we need 10 output neurons.
3 # Rule 2d: Output layer MUST use 'softmax'
4 model.add(Dense(10, activation='softmax'))
```

Listing 10: The output layer

The Output Layer is the layer that actually produces the final prediction. It classifies the results from the previous hidden layers into categories. The categories are actually the digits 0-9.



Figure 6: The softmax function takes a score vector s and a value of the softmax-parameter as input and returns a probability vector.

The *Softmax* function takes a vector of scores and maps it to a vector of probabilities. In our code, the function converts the raw scores (logits) from the output neurons into probabilities that sum up to exactly 1.0 (or 100%). For example, if the network sees a '7', *Softmax* might output a value of 0.95 for the digit 7 and very low values for the others. [7]

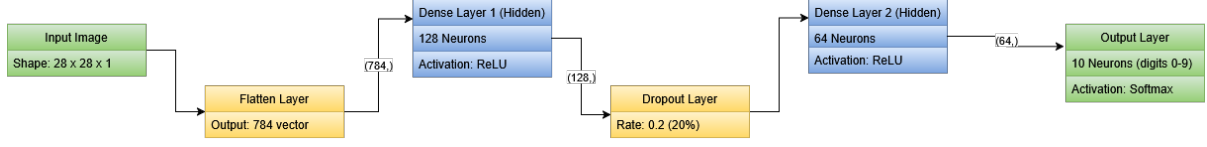


Figure 7: Architecture of the Proposed Deep Neural Network (DNN).

Input and Output Layers are coloured green.

Flatten and Dropout Layers are coloured yellow.

Dense Hidden Layers are coloured blue.

3.2.2 Convolutional Neural Network (CNN)

CNNs stands for Convolutional Neural Networks. They are similar to Traditional Neural Networks, although they are commonly used for Image Classification tasks, just like our project. Traditional Neural Networks tend to struggle with the computational complexity required to compute image data. The MNIST dataset is a dataset that can be handled by a simple **ANN**, due to its small size. In real life examples, the size of the input data can never be that small, so **CNNs** are specifically designed to handle larger input data.

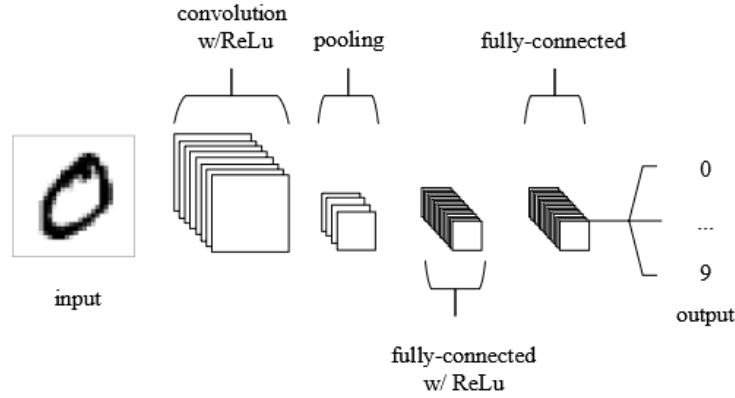


Figure 8: A simple CNN architecture, comprised of just five layers

The primary difference between a Convolutional Neural Network (CNN) and a standard Deep Neural Network (DNN) lies in how the neurons are connected. In a standard DNN, every neuron is fully connected to all neurons in the previous layer. In contrast, a CNN neuron is only connected to a small, localized region of the input. Furthermore, the layers within a CNN are comprised of neurons organized into three dimensions: **the spatial dimensions** (height and width) and **the depth** (feature channels). Overall, CNN architectures are typically built using three main types of layers. [8]

The basic functionality of the proposed CNN architecture can be broken down into four key stages:

1. **Input Layer:** As found in other forms of ANNs, this layer holds the raw pixel values of the image. For our specific task, the input corresponds to the $28 \times 28 \times 1$ grayscale images of the MNIST dataset.

2. **Convolutional Layer & Activation:** This layer acts like a set of scanning filters. It calculates the dot product between the filters' weights and local regions of the input image, allowing the network to detect spatial patterns such as edges and curves. Immediately after, an element-wise activation function is applied. In our model, we use the Rectified Linear Unit (*ReLU*), which simply replaces all negative values with zero, enabling the network to learn complex patterns faster and more efficiently than older functions like the Sigmoid.
3. **Pooling Layer:** This layer performs spatial downsampling (e.g., Max Pooling) along the height and width of the data. Its purpose is to retain only the most prominent features while significantly reducing the spatial dimensions, thereby decreasing the computational load and the risk of overfitting.
4. **Fully-Connected Layers:** Once the convolutional and pooling layers have successfully extracted the meaningful spatial features, the data is flattened into a one-dimensional vector. The fully-connected layers then perform the same duties found in standard DNNs, processing these extracted features to finally produce the class scores (probabilities) used for the classification of the digits.

We are going to show a sample of the code that was used to build the CNN model. A frame of the code will be shown, followed by an explanation.

First Convolutional Layer

```
1 # 1. First Convolutional Layer
2 # It uses 6 filters of size 3x3. It reads the 28x28x1 image.
3 # Rule 2b: Use 'relu' activation
4 model.add(Conv2D(6, kernel_size=(3, 3), activation='relu', input_shape
    =(28, 28, 1)))
```

Listing 11: First Convolutional Layer

This is our First Convolutional Layer. It uses 6 small filters (kernels) of size 3x3 to scan the image. These filters learn to detect low-level features like edges or corners.

First Pooling Layer

```
1 # 2. First Pooling Layer
2 # Shrinks the image by taking the maximum value in every 2x2 window
3 model.add(MaxPooling2D(pool_size=(2, 2)))
```

Listing 12: First Pooling Layer

This layer reduces the spatial dimensions (width and height) of the image by half. It takes the maximum value in every 2x2 pixel block, which helps make the model faster and less sensitive to the exact position of features.

Second Convolutional Layer

```
1 # 3. Second Convolutional Layer
2 # Now we use 12 filters of size 3x3 to find more complex patterns
3 model.add(Conv2D(12, kernel_size=(3, 3), activation='relu'))
```

Listing 13: Second Convolutional Layer

A second set of layers is added, this time with 12 filters to learn more complex patterns from the features identified by the first layer.

Second Pooling Layer

```
1 # 4. Second Pooling Layer
2 # Shrinks the feature maps again
3 model.add(MaxPooling2D(pool_size=(2, 2)))
```

Listing 14: Second Pooling Layer

This layer reduces the spatial dimensions (width and height) of the image by half. It takes the maximum value in every 2x2 pixel block, which helps make the model faster and less sensitive to the exact position of features.

Flatten Layer

```
1 # 5. Flatten Layer
2 # Now that the CNN has found the patterns, we flatten the output
3 # to feed it into a standard Dense network for the final decision
4 model.add(Flatten())
```

Listing 15: Flatten Layer

Convolutional layers output 3D feature maps. To connect them to a standard Dense (fully connected) layer, we must 'flatten' them into a single long 1D list of numbers.

Hidden Dense Layer

```
1 # 6. Hidden Dense Layer
2 model.add(Dense(128, activation='relu'))
```

Listing 16: Hidden Dense Layer

This is a hidden layer with 128 neurons that performs the final classification reasoning.

Dropout Layer

```
1 # 7. Dropout Layer
2 # Added to satisfy Rule 2c: use at least one dropout layer.
3 model.add(Dropout(0.25))
```

Listing 17: Dropout Layer

This randomly 'turns off' 25% of the neurons during training. It is a regularization technique used to prevent the model from overfitting (memorizing) the training data.

Output Layer

```
1 # 8. Output Layer
2 # 10 output digits (0-9). Rule 2d: Output layer MUST use 'softmax'
```

```
3 model.add(Dense(10, activation='softmax'))
```

Listing 18: Output Layer

The output layer. It has 10 neurons (one for each digit 0-9). The *Softmax* function turns the output into probabilities that sum up to 1.0.

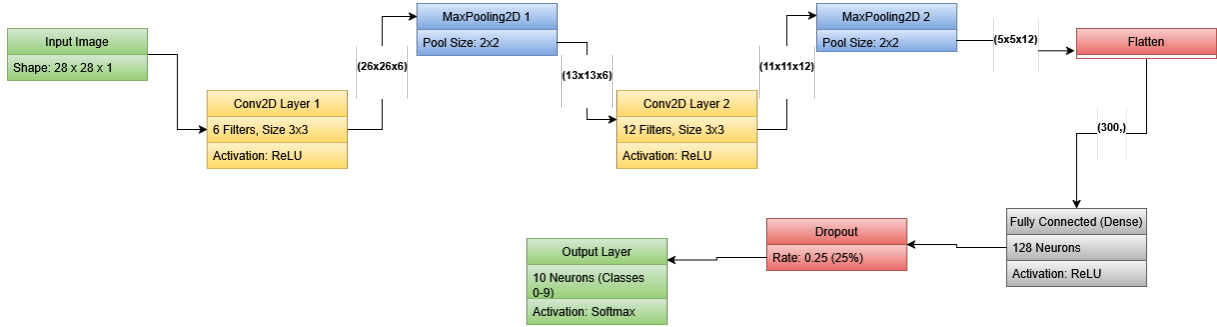


Figure 9: Architecture of the Proposed Convolutional Neural Network (CNN) highlighting the feature map dimensions at each stage.

Input and Output Layers are coloured green.

Conv2D Layers are coloured yellow.

MaxPooling2D Layers are coloured blue.

Flatten and Dropout Layers are coloured pink.

The Dense Layer is coloured gray.

3.3 Transfer Learning Methodology

Transfer Learning is a powerful machine learning technique where a model developed for one specific task is reused as the starting point for a model on a second, related task. In the context of deep learning, Yosinski et al. demonstrated that the initial layers of a Convolutional Neural Network learn generalized features (such as edges and curves) that are applicable to many different datasets, whereas the final layers learn highly specific details. [9]

Therefore, instead of training a model entirely from scratch—which requires massive amounts of data and computational power—we can leverage these pre-trained weights. According to the official Keras guidelines, the standard transfer learning workflow consists of removing the original classification head, freezing the base layers to retain their generalized knowledge, adding a new output layer, and finally fine-tuning the network carefully. [10]

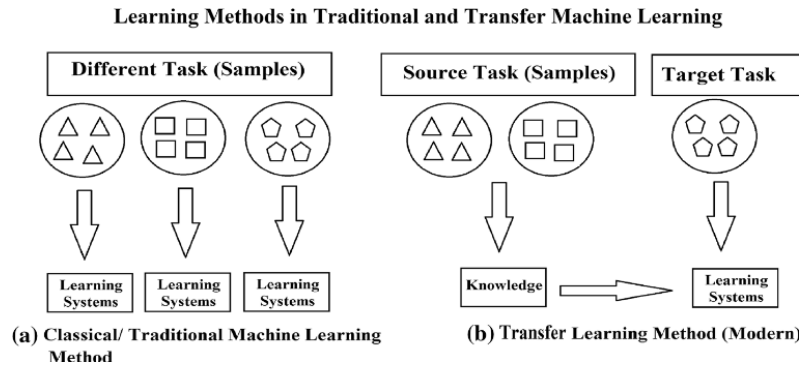


Figure 10: Traditional/Classical ML vs Transfer Learning

In Figure 10, we can see how Transfer Learning enhances learning by creating a relation between previous tasks and the target task, providing logical, faster, and better solutions. [11]

At this point, the Transfer Learning Model Builder, will be introduced, as a sample of code.

The pre-trained model path

```
1 # 1. Load the pre-trained model (trained on MNIST from Question 1)
2 base_model = keras.models.load_model(saved_model_path)
```

Listing 19: Loading the pre-trained model

This block of code, loads the pre-trained model. The path is saved on the `saved_model_path` variable.

An empty Sequential Model

```
1 # 2. Create a new, empty Sequential model
2 transfer_model = Sequential()
```

Listing 20: An empty Sequential Model

We create a new Sequential Model, so that we can keep only the necessary structure.

Looping through layers

```
1 # 3. Loop through all layers of the loaded model EXCEPT the last one
  ([::-1])
2 for layer in base_model.layers[::-1]:
3     # Freeze the layer so its pre-trained weights don't change
4     layer.trainable = False
5     # Add the frozen layer to our new model
6     transfer_model.add(layer)
```

Listing 21: Loop that goes through all the layers except the last one

This loop goes through the layers of the pre-loaded model and copies them into the empty Sequential Model, that was declared before. The last layer is excluded because it was exclusively used for the data of the previous task. Freezing the layers, guarantees that the pre-trained weights won't change at the training process.

Classification Layer

```
1 # 4. Add a fresh classification layer for Fashion-MNIST (10 clothing
   classes)
2 transfer_model.add(Dense(10, activation='softmax'))
```

Listing 22: A classification layer with the Softmax function

This is the final output layer. It uses the *Softmax* function as the activation function, to give a probability for each one of the 10 clothing classes.

4 Training Process and Evaluation Strategy

4.1 Task 1 - The MNIST Dataset

To test our models fairly and prevent them from just memorizing the data (a problem known as overfitting), we organized the training process step by step. First, the MNIST dataset is split into two main parts: 60,000 images that are used for training the models, and 10,000 new images that are used only for testing them at the very end.

Instead of relying on a single train-validation split, we employed the **Stratified K-Fold Cross-Validation** technique. Specifically, the 60,000 training samples were partitioned into $K = 6$ equally sized folds. The term "stratified" guarantees that each fold maintains the same proportion of digits (0-9) as the original dataset, preventing class imbalance issues during training.

The iterative evaluation process was executed as follows:

- For each of the 6 iterations, one fold (10,000 samples) was held out as the **Validation Set**, while the remaining 5 folds (50,000 samples) constituted the **Training Set**.
- Both the DNN and CNN models were initialized from scratch and trained on the 50,000 samples.
- At the end of every epoch, the models were evaluated internally using the 10,000 validation samples to monitor the learning curve and calculate the validation loss.
- Upon the completion of the training loop for a specific fold, the fully trained model was evaluated against the 10,000 samples of the global **Test Set** to record the final classification metrics (Accuracy, Precision, Recall, and F1-score).

Hyperparameters Configuration

Both models were compiled and trained using the following uniform hyperparameter settings to ensure a fair comparison:

- **Loss Function:** Categorical Crossentropy, which is the standard mathematical approach for multi-class classification problems involving one-hot encoded labels.
- **Optimizer:** The Adam optimizer, chosen for its adaptive learning rate capabilities and computational efficiency.
- **Batch Size:** 128, allowing the models to process data in chunks rather than one-by-one, which stabilizes the gradient updates and speeds up training.
- **Epochs:** 5 epochs per fold, providing a sufficient balance between learning the underlying patterns and maintaining reasonable training times for 12 total models.

4.2 Task 2 - The Fashion-MNIST Dataset

Following the established methodology on Transfer Learning, our goal for the second task of this assignment was to test if TL is actually helpful for our specific models. To prove this, we compared two different approaches on the Fashion-MNIST dataset:

4.2.1 Training from Scratch (The Baseline)

A DNN and a CNN were created with the exact same architectures that were used to train the MNIST Dataset. These models had no prior memory, so they were used from zero to train the Fashion-MNIST dataset, allowing comparison of performance and generalization between tasks. The architecture of the models are available here: [DNN-3.2.1](#), [CNN-3.2.2](#)

Specifically, both networks were trained for 150 epochs. Because they were initialized with completely random weights and lacked any pre-existing knowledge, this prolonged training duration was strictly necessary to allow the models to learn the intricate patterns and shapes of the clothing categories from scratch.

4.2.2 Transfer Learning Approach

To implement this strategy, we initially loaded the best-performing DNN and CNN models (saved as .h5 files) that were successfully trained on the MNIST dataset during the first task. Since the original models were configured to recognize digits, their final classification layer was removed. In its place, a newly initialized Dense layer with 10 neurons and a Softmax activation function was appended, exactly corresponding to the 10 distinct clothing categories of the Fashion-MNIST dataset.

Phase 1 (Freezing)

During the first phase of training, all the transferred base layers from the original models were "frozen" (i.e., their weights were set to be non-trainable). The network was then trained in this state for 40 epochs. This step is crucial because it allows the newly added classification layer to adapt to the Fashion-MNIST data and adjust its random weights, without disrupting or destroying the valuable feature extraction capabilities (such as detecting edges and curves) already stored in the frozen layers.

Phase 2 (Fine-Tuning)

After the new classification layer was adequately trained, all layers of the network were "unfrozen" (i.e., set back to trainable). The entire architecture was then trained end-to-end for an additional 20 epochs. This final phase, known as fine-tuning, permits the network to make very small, careful adjustments to its older, pre-trained weights. By doing so, the model perfectly aligns its previously learned knowledge of basic shapes with the specific, complex details of the clothing items.

Cross-Validation Strategy

To guarantee that our results are robust and not dependent on a lucky data split, all experiments in this second task were evaluated using a **Stratified 6-Fold Cross-Validation** approach:

- **Train and Validation Sets:** For every iteration (fold), the initial dataset of 60,000 images was dynamically split. Exactly 50,000 images were used strictly as the *Training Set* to teach the model, while the remaining 10,000 images were held

out as the *Validation Set*. This internal validation set was used at the end of each epoch to monitor the learning progress and ensure the model was not overfitting.

- **Final Test Set:** Only after a model finished its entire training loop on a specific fold, it was finally evaluated on the global, completely unseen *Test Set* (10,000 samples) to record the ultimate classification scores.

Hyperparameters Configuration

To ensure a fair and consistent comparison between the baseline models and the transfer learning models, the fundamental hyperparameters were kept identical across all experiments:

- **Optimizer & Loss Function:** The Adam optimizer and the Categorical Crossentropy loss function were utilized universally.
- **Batch Size:** Set to 128 to ensure stable gradient updates and computational efficiency.
- **Training Epochs:** The number of epochs was carefully adjusted depending on the specific phase of the training process:
 - Training from Scratch (Baseline): 150 epochs.
 - Transfer Learning - Phase 1 (Frozen): 40 epochs.
 - Transfer Learning - Phase 2 (Fine-Tuning): 20 epochs.

5 Experimental Results

5.1 Part 1: Architecture Comparison (MNIST)

To evaluate the effectiveness of the proposed models, we analyzed their performance on the completely unseen test set (10,000 samples). Since we employed a 6-Fold cross-validation strategy, the final metrics were calculated by averaging the scores across all six independent iterations.

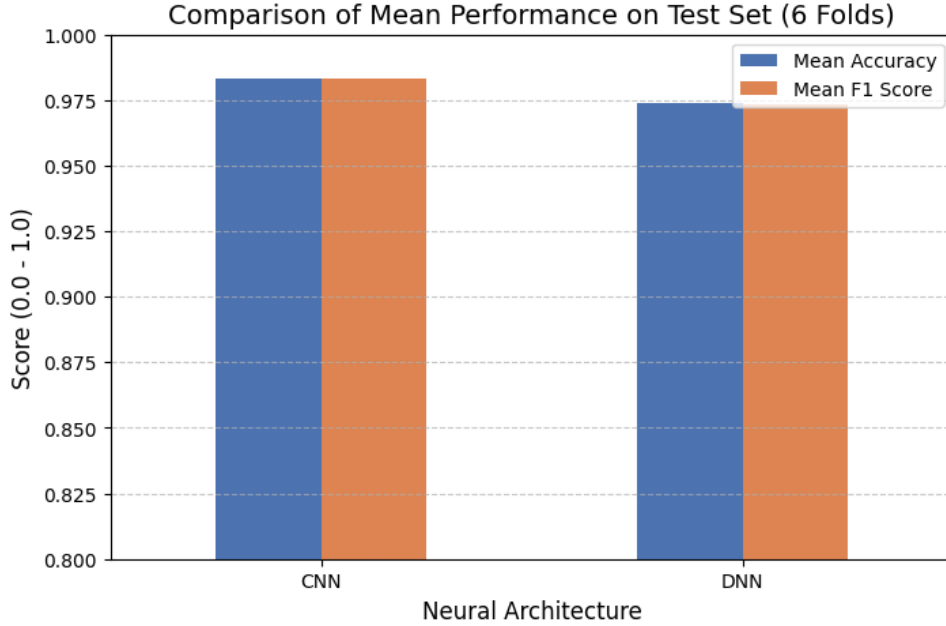


Figure 11: Average Accuracy and F1-Score of the DNN and CNN architectures on the MNIST test set across 6 folds.

As clearly illustrated in Figure 11, **the Convolutional Neural Network (CNN) outperforms the standard Deep Neural Network (DNN)** across all classification metrics. While the DNN achieves a respectable performance, the CNN demonstrates superior accuracy and stability.

Explanation

- **DNN:** The reason behind this difference in performance lies in how each network processes the input image. When using the DNN, the image is immediately flattened into a single, one-dimensional line of pixels. Because of this, the network completely loses the **spatial relationships** between the pixels—it essentially forgets which pixel was next to which, making it much harder to recognize the actual shape of the digit.
- **CNN:** On the other hand, the CNN processes the image in its original 2D grid shape. Its convolutional layers act like small magnifying glasses that scan the image to find specific local patterns, such as the round loop of an '8' or the sharp angle of a '7'. Because the CNN can actually "see" and understand these structural features, it is naturally much more effective and accurate for computer vision tasks than a standard dense network.

5.2 Part 2: Architecture Comparison (Fashion-MNIST)

For the second task, we evaluated whether transferring knowledge from the MNIST dataset could improve performance on the more complex Fashion-MNIST dataset. Figure 12 presents the average Accuracy and F1-score on the Test set for all four models across the 6 folds.

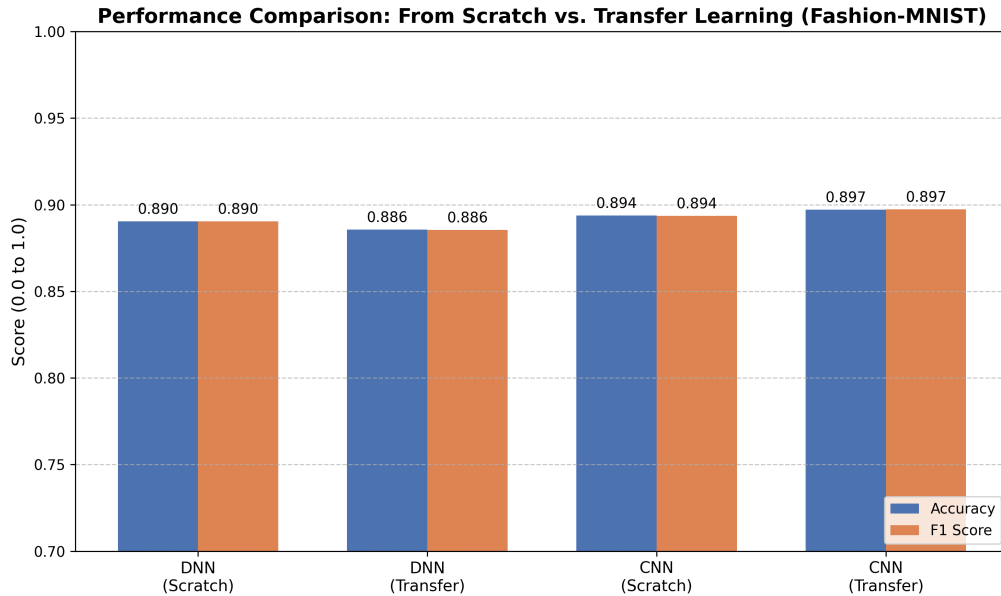


Figure 12: Performance comparison between models trained from scratch and models utilizing Transfer Learning on the Fashion-MNIST test set.

CNN with Transfer Learning achieved the highest overall performance, successfully outperforming both the models trained from scratch and the standard DNN. This outcome confirms the utility of transfer learning in computer vision tasks.

Explanation

- **CNN vs. DNN:** The CNN naturally excels at processing images because its convolutional filters can "see" the 2D shapes of the clothing items (like the sleeves of a shirt or the heel of a shoe), whereas the DNN simply flattens the image and loses this spatial information.
- **Transfer Learning vs. Scratch:** The models trained "from scratch" had to start with random weights, meaning they had to figure out what a basic line or curve looks like before they could even begin to recognize a dress or a sneaker. On the other hand, the Transfer Learning CNN already knew how to detect basic geometric shapes (learned from the digits 0-9). By freezing those layers and only fine-tuning them later, the network simply reused its old knowledge to learn the new clothing patterns much faster and with greater accuracy.

6 Conclusions

Purpose of this project

The primary purpose of this project was to explore the fundamentals of Deep Learning and Computer Vision. State-of-the-art technologies, such as Artificial Neural Networks, were utilized to achieve high accuracy in Image Classification and Pattern Recognition tasks.

Specifically, DNN and CNN architectures were implemented and compared to determine the most effective model for classifying the MNIST dataset, while Transfer Learning techniques were further evaluated on the more complex Fashion-MNIST dataset.

The First Task

In the first task of the project, we were assigned to determine which Neural Network architecture is more suitable for Computer Vision tasks. A standard DNN and a CNN were implemented to classify handwritten digits from the MNIST dataset.

The results showed that the CNN model performed better, due to its ability to process images in a 2D shape and detect visual patterns like edges and curves. The performance gap between the two models was not significant, since the DNN model also achieved high accuracy, but it was not capable of outperforming the CNN.

The Second Task

In the second task, the goal was to evaluate if Transfer Learning could improve model performance on a new, more complex dataset. To achieve this, we compared models trained completely from scratch against our pre-trained models from the first task, adapting them to classify the Fashion-MNIST dataset.

The results highlighted that the CNN with Transfer Learning was the most effective model. By reusing its pre-trained weights, the network already knew how to detect basic geometric features, which helped it learn the new clothing categories much faster. This proves that transferring knowledge between visually similar tasks is a highly beneficial strategy, yielding better accuracy than training a network entirely from zero.

Limitations and further improvements

A limitation of our methodology was definitely the computational time required for the experiments of the second task. Training for exactly 150 epochs demanded a lot of computational resources, which means that larger datasets could possibly need paid computational resources on the cloud.

For future improvements, an "Early Stopping" mechanism could be implemented to automatically halt the training process once the model stops learning. Furthermore, evaluating these architectures on more complex, colored image datasets (such as CIFAR-10) would be a great next step to fully understand their capabilities and limits in real-world scenarios.

References

- [1] “MNIST Dataset : Practical Applications Using Keras and PyTorch,” May 2024. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/mnist-dataset/>
- [2] zalando research, “GitHub - zalando research/fashion-mnist: A MNIST-like fashion product database. Benchmark.” [Online]. Available: <https://github.com/zalando research/fashion-mnist>
- [3] K. Team, “Keras documentation: MNIST digits classification dataset.” [Online]. Available: <https://keras.io/api/datasets/mnist/>
- [4] R. Mohanasundaram, A. S. Malhotra, R. Arun, and P. Periasamy, “Deep Learning and Semi-Supervised and Transfer Learning Algorithms for Medical Imaging,” in *Deep Learning and Parallel Computing Environment for Bioengineering Systems*. Academic Press, Jan. 2019, pp. 139–151. [Online]. Available: <https://www.sciencedirect.com/science/chapter/edited-volume/pii/B9780128167182000154>
- [5] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, Sep. 2017. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8114708>
- [6] Y. Bai, “RELU-Function and Derived Function Review,” *SHS Web of Conferences*, vol. 144, p. 02006, 2022. [Online]. Available: https://www.shs-conferences.org/articles/shsconf/abs/2022/14/shsconf_stehf2022_02006/shsconf_stehf2022_02006.html
- [7] M. Franke and J. Degen, “The softmax function: Properties, motivation, and interpretation,” Sep. 2023. [Online]. Available: https://osf.io/vsw47_v1
- [8] K. O’Shea and R. Nash, “An Introduction to Convolutional Neural Networks,” Dec. 2015, arXiv:1511.08458 [cs]. [Online]. Available: <http://arxiv.org/abs/1511.08458>
- [9] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” in *Advances in Neural Information Processing Systems*, vol. 27. Curran Associates, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/hash/532a2f85b6977104bc93f8580abbb330-Abstract.html
- [10] K. Team, “Keras documentation: Transfer learning & fine-tuning.” [Online]. Available: https://keras.io/guides/transfer_learning/
- [11] A. Hosna, E. Merry, J. Gyalmo, Z. Alom, Z. Aung, and M. A. Azim, “Transfer learning: a friendly introduction,” *Journal of Big Data*, vol. 9, no. 1, p. 102, Oct. 2022. [Online]. Available: <https://doi.org/10.1186/s40537-022-00652-w>